

UNIVERSIDADE FEDERAL DA PARAÍBA
CENTRO DE INFORMÁTICA
CURSO DE CIÊNCIA DA COMPUTAÇÃO

Michel Diniz Medeiros



loaderEFD: Sistema de Ingestão de Escrituração Fiscal
Digital baseado em regras declarativas
Relatório técnico

João Pessoa, PB, 2026

Michel Diniz Medeiros

loaderEFD: Sistema de Ingestão de Escrituração Fiscal
Digital baseado em regras declarativas
Relatório técnico

Monografia apresentada ao curso Ciência da Computação do Centro de Informática, da Universidade Federal da Paraíba, como requisito para a obtenção do grau de Bacharel em Ciência da Computação

Orientador: Marcelo Iury de Sousa Oliveira

Agosto de 2026

Ficha catalográfica:

Será impressa no verso da folha de rosto e não deverá ser contada.

Se não houver biblioteca, deixar em branco.



CENTRO DE INFORMÁTICA
UNIVERSIDADE FEDERAL DA PARAÍBA

Trabalho de Conclusão de Curso de Ciência da Computação intitulado ***loaderEFD: Sistema de Ingestão de Escrituração Fiscal Digital baseado em regras declarativas*** de autoria de Michel Diniz Medeiros, aprovada pela banca examinadora constituída pelos seguintes professores:

Prof. Dr. Marcelo Iury de Sousa Oliveira
UFPB

Prof. Dr. Raoni Kulesza
UFPB

Prof.^a Dr.^a Thaís Gaudencio do Rêgo
UFPB

João Pessoa, 26 de agosto de 2026

*“Human beings have a remarkable ability to
accept the abnormal and make it normal.”
— Andy Weir, Project Hail Mary*

AGRADECIMENTOS

Aos meus pais, Maria e Marinaldo, por acreditarem que tudo isso valia a pena e me motivarem a seguir em frente.

Aos meus tios, Inácio e Valdelice, e ao meu primo, Edmilson, que me acolheram em seu lar e tornaram essa caminhada possível.

A minha amiga e irmã do peito, Laura, que esteve comigo ao longo de toda essa graduação, me oferecendo um ombro amigo e um abraço solidário com o qual sempre puder contar.

A meu amigo Lutero, que compartilhou comigo discussões, disciplinas e ideias que me fizeram rir, refletir e progredir ao longo desses anos.

A meu amigo Arthur, que compartilhou comigo boas músicas, ótimas conversas e histórias incríveis que serão boas memórias que vão estar comigo para sempre.

A meu amigo Nicolas, que compartilhou comigo boas horas jogando, diálogos profundos e rasos e uma companhia incrível, que me mostrou que nem só de estudos se constrói uma graduação.

A minha namorada, Paloma, que iluminou minha vida com seu amor e me proporcionou dias mais leves e alegres.

Aos meus alunos da monitoria, em especial Deykson, pela oportunidade de aprender com vocês e de vivenciar as maravilhas de ensinar.

Aos docentes da UFPB que participaram da minha formação acadêmica. Um agradecimento especial ao Prof. Dr. Marcelo Iury de Sousa Oliveira, pela orientação e amizade ao longo de toda minha estadia no ARIA.

Por fim, quero agradecer a todos pela parceria e amizade que tornaram essa aventura inesquecível para muito além do que eu havia sonhado.

RESUMO

O processamento e a auditoria de documentos da Escrituração Fiscal Digital de ICMS e IPI (EFD ICMS IPI) enfrentam gargalos operacionais no setor público e privado. A estrutura em texto plano hierárquico desses arquivos, a alta volatilidade dos leiautes legais do SPED e a ausência de uma especificação oficial legível por máquina tornam os sistemas tradicionais baseados em código fixo inflexíveis e propensos a falhas. Para solucionar esse problema, o presente trabalho desenvolve um *pipeline* de ingestão dinâmico e orientado metadados. A arquitetura elaborada extrai as regras de negócio da documentação fiscal e as estrutura em arquivos de configuração JSON, desacoplando o processamento do código-fonte. A solução é composta por um Orquestrador que distribui a carga de trabalho em lotes e gerencia a execução paralela de trabalhadores dinâmicos em memória, responsáveis por interpretar a hierarquia do texto e realizar a persistência estruturada em um banco de dados relacional *PostgreSQL*. A validação empírica atestou zero perda informacional e a manutenção da integridade referencial por meio do teste de reconstrução reversa dos dados originais. A solução também demonstrou alta resiliência no tratamento de arquivos corrompidos e garantiu a atomicidade das operações no banco. Conclui-se que a abordagem por metadados confere alta flexibilidade às mudanças da legislação tributária, e as limitações encontradas oferecem largo espaço para desenvolvimento e expansões futuras da ferramenta.

Palavras-chave: EFD ICMS IPI. SPED. Engenharia de Dados. Ingestão de Dados. Arquitetura Orientada a Metadados. Banco de Dados Relacional.

ABSTRACT

The processing and auditing of Digital Fiscal Bookkeeping for ICMS and IPI (EFD ICMS IPI) documents face operational bottlenecks in both the public and private sectors. The hierarchical plain text structure of these files, the high volatility of legal SPED layouts, and the absence of an official machine-readable specification render traditional hard-coded systems inflexible and error-prone. To solve this problem, this study develops a dynamic, metadata-driven ingestion pipeline. The proposed architecture extracts business rules from tax documentation and structures them into JSON configuration files, decoupling data processing from the source code. The solution consists of an Orchestrator that distributes the workload into batches and manages the parallel execution of in-memory dynamic workers, which parse the textual hierarchy and perform structured persistence into a *PostgreSQL* relational database. Empirical validation demonstrated zero information loss and preserved referential integrity through reverse reconstruction tests of the original data. The solution also showed high resilience in handling corrupted files and ensured database transaction atomicity. It is concluded that the metadata-driven approach provides high flexibility regarding changes in tax legislation, while the identified limitations offer substantial room for future development and tool expansions.

Keywords: EFD ICMS IPI. SPED. Data Engineering. Data Ingestion. Metadata-Driven Architecture. Relational Database.

LISTA DE FIGURAS

1	Diagrama de arquitetura da solução de ingestão para arquivos EFD ICMS IPI. .	28
2	Diagrama exemplificando a lógica do despachante.	31
3	Diagrama exemplificando a lógica de dispersão das chaves primárias dos pais. .	33
4	Distribuição dos tempos de processamento dos <i>Workers</i> em minutos em cada instância de execução.	44

LISTA DE TABELAS

1	Tempo bruto de execução por volumetria	42
2	Estatísticas por lote (<i>Baseline</i>)	42

LISTA DE QUADROS

1	Exemplo de documento EFD	20
2	Exemplo de modelo JSON de regras de metadados para a EFD ICMS IPI . . .	21
3	Especificações do ambiente computacional	35
4	Exemplo de log de erro de <i>parsing</i>	38
5	Exemplo de log de erro durante a carga	38

LISTA DE ABREVIATURAS

ABNT — Associação Brasileira de Normas Técnicas

ARIA — *Artificial Intelligence Applications Laboratory*

DF-e — Documento Fiscal Eletrônico

DFS — Busca em Profundidade (do inglês *Depth-First Search*)

EFD ICMS IPI — Escrituração Fiscal Digital de ICMS e IPI

E/S — Entrada e Saída

ICMS — Imposto sobre Operações relativas à Circulação de Mercadorias e sobre Prestações de Serviços de Transporte Interestadual e Intermunicipal e de Comunicação

IPI — Imposto sobre Produtos Industrializados

JSON — Notação de Objetos do JavaScript (do inglês *JavaScript Object Notation*)

JSONB — JSON Binário (do inglês *Binary JSON*)

NF-e — Nota Fiscal Eletrônica

NUMA — Acesso Não-Uniforme à Memória (do inglês *Non-Uniform Memory Access*)

RAM — Memória de Acesso Aleatório (do inglês *Random Access Memory*)

SEFAZ — Secretaria da Fazenda

SGBD — Sistema Gerenciador de Banco de Dados

SPED — Sistema Público de Escrituração Digital

PB — Paraíba

PDF — Formato de Documento Portátil (do inglês *Portable Document Format*)

UFPB — Universidade Federal da Paraíba

XML — Linguagem de Marcação Extendida (do inglês *Extensible Markup Language*)

XSD — Definição de *schema* XML (do inglês *XML Schema Definition*)

SUMÁRIO

1	INTRODUÇÃO	14
1.1	Objetivo geral	15
1.2	Objetivos específicos	15
1.3	Estrutura do relatório técnico	16
2	Trabalhos relacionados	17
2.1	<i>Pipelines</i> de Ingestão de Dados Adaptativos	17
2.2	Armazenamento de documentos fiscais	17
3	<i>Pipeline</i> Baseado em Metadados	19
3.1	EFD ICMS IPI e seu Modelo Semiestruturado	19
3.2	Especificação Estruturada e Modelagem de Regras (Metadados)	20
3.3	Arquitetura da Solução de Ingestão de Dados	27
3.3.1	Orquestrador	27
3.3.2	<i>Worker</i>	29
3.3.3	<i>Parser</i>	29
3.3.4	<i>Loader</i>	30
4	APRESENTAÇÃO E ANÁLISE DOS RESULTADOS	35
4.1	Ambiente de execução e hardware utilizado	35
4.2	Metodologia de testes	36
4.3	Limitações encontradas após a modelagem	39
4.3.1	Limitações do Tipo JSONB com Grandes Volumes	39
4.3.2	Degradação da inserção	40

4.3.3	Documentação conflitante	41
4.4	Métricas Operacionais Preliminares e Linha de Base	41
4.5	Síntese dos Resultados	43
5	CONCLUSÕES E TRABALHOS FUTUROS	45
5.1	Trabalhos Futuros	46
6	DECLARAÇÃO SOBRE USO DE INTELIGÊNCIA ARTIFICIAL	47
	REFERÊNCIAS	47

1 INTRODUÇÃO

Estabelecido em 2007 com o Decreto nº 6.022/2007 [1], o Sistema Público de Escrituração Digital, abreviado como SPED, é um sistema que busca modernizar a interação entre contribuintes e a administração tributária pública encarregada de regulamentar, arrecadar, fiscalizar e auditar a prestação de contas e o pagamento dos tributos, administração essa denominada ao longo desse documento como *fisco*. O SPED executa essa modernização através da informatização das obrigações acessórias, bem como a utilização da certificação digital para garantir a validade jurídica dos documentos eletrônicos [2]. Um dos módulos desse sistema é a Escrituração Fiscal Digital de ICMS — Imposto sobre Operações relativas à Circulação de Mercadorias e sobre Prestações de Serviços de Transporte Interestadual e Intermunicipal e de Comunicação — e IPI — Imposto sobre Produtos Industrializados —, abreviada como EFD ICMS IPI, que opera por meio de um arquivo digital, cuja emissão é obrigatória para contribuintes desses impostos, contendo registros de apuração de impostos, documentos fiscais e outras informações de interesse da Secretaria da Receita Federal, dos fiscos estaduais e do Distrito Federal [3].

O documento EFD ICMS IPI deve ser escriturado pelo contribuinte, validado se utilizando do programa especificado pelos fiscos das unidades federadas e pelo fisco federal, e submetido a programa validador, disponibilizado em sítio digital do SPED, para validação de conteúdo, assinatura digital e transmissão [4]. A responsabilidade pela governança e custódia desses dados recai diretamente sobre a Administração Tributária. Sob a égide do Decreto nº 6.022/2007 [1], o sistema é explicitamente definido em seu Artigo 2º como “[...] *instrumento que unifica as atividades de recepção, validação, armazenamento e autenticação de livros e documentos que integram a escrituração contábil e fiscal dos empresários e das pessoas jurídicas, inclusive imunes ou isentas, mediante fluxo único, computadorizado, de informações.*”. Portanto, a manutenção íntegra e perpétua desse acervo digital configura um dever legal imperativo do Estado, essencial para o exercício pleno da fiscalização soberana.

Além disso, no âmbito da Lei 5.172/1966 [5], em seu Artigo 150º, Parágrafo 4º, é explicitado que “*Se a lei não fixar prazo a homologação, será êle de cinco anos, a contar da ocorrência do fato gerador; expirado êsse prazo sem que a Fazenda Pública se tenha pronunciado, considera-se homologado o lançamento e definitivamente extinto o crédito, salvo se comprovada a ocorrência de dolo, fraude ou simulação*”, sendo também reforçado pelo artigo 173º desse mesmo decreto, que afirma “*O direito de a Fazenda Pública constituir o crédito tributário extingue-se após 5 (cinco) anos, contados: I — do primeiro dia do exercício seguinte àquele em que o lançamento poderia ter sido efetuado; II — da data em que se tornar definitiva a decisão que houver anulado, por vício formal, o lançamento anteriormente efetuado*”, de modo que além do armazenamento de tais documentos, é de interesse do fisco realizar auditorias sobre esses documentos em tempo hábil, para evitar fraudes fiscais e homologação involuntárias nos

termos desses decretos, o que pode gerar prejuízos bilionários aos cofres públicos.

Contudo, a efetivação dessas auditorias esbarra em diversos desafios de engenharia de software e processamento de dados. Os arquivos da EFD ICMS IPI são semiestruturados, apresentando uma limitação física de blocos, mas deixando de fora a hierarquia entre os registros. Essa hierarquia, bem como outras regras de formatação, formam o que é denominado de *leiaute*, o qual sofre atualizações frequentes por parte do fisco [6]. A ingestão desse volume massivo de dados semiestruturados para sistemas gerenciadores de bancos de dados (SGBD) relacionais — utilizando abordagens tradicionais de código fixo — resulta em alto custo de manutenção e dificuldades no mapeamento de entidades, como limitações na manipulação e de cruzamento nesses grandes volumes de dados.

Com este cenário estabelecido, o presente trabalho se propõe a desenvolver um *pipeline* de ingestão dinâmico e orientado a metadados para o processamento de arquivos EFD ICMS IPI. A solução busca abstrair as regras de extração do código-fonte, por meio de arquivos de configuração em formato JSON (Notação de Objetos do JavaScript, tradução livre do inglês *JavaScript Object Notation*), permitindo que a infraestrutura do fisco processe e armazene os dados fiscais de forma confiável e resiliente às constantes mudanças de *leiaute*.

1.1 Objetivo geral

Desenvolver uma arquitetura de ingestão e processamento de dados orientada a metadados para arquivos EFD ICMS IPI, visando permitir o armazenamento estruturado e facilitar o cruzamento de dados fiscais.

1.2 Objetivos específicos

Para que o objetivo geral seja alcançado, foram traçados os seguintes objetivos específicos:

- Determinar, dentre as diversas regras especificadas na documentação oficial [4, 6], quais são relevantes para o nosso objetivo;
- Mapear e estruturar as regras de negócio relevantes dos *leiautes* do SPED Fiscal utilizando arquivos de metadados estruturado em JSON;
- Implementar uma ferramenta que seja capaz de ler, extrair e atribuir a estrutura do *leiaute* aos blocos de texto da EFD ICMS IPI de forma dinâmica, *i.e.*, sem dependência de código fixo;
- Validação da integridade dos dados.

1.3 Estrutura do relatório técnico

Para uma melhor compreensão e fluidez da leitura, as próximas páginas se encontram divididas em um total de 5 seções, além das referências bibliográficas. A Seção 2 aborda trabalhos semelhantes ao presente. A Seção 3, por sua vez, detalha com mais riqueza o contexto do problema, usando isso como base para apresentar a solução elaborada. A Seção 4 expõe os resultados obtidos e faz uma análise de cada um deles, cobrindo tópicos específicos que foram considerados relevantes. A Seção 5 reúne as considerações finais, as contribuições da pesquisa e as sugestões para trabalhos futuros. Por fim, a Seção 6 apresenta a declaração formal sobre o uso de ferramentas de Inteligência Artificial ao longo da elaboração deste documento.

2 Trabalhos relacionados

Esta seção apresenta uma revisão da literatura fundamentada em dois eixos centrais para o posicionamento deste trabalho: a adoção de *pipelines* de ingestão adaptativos baseados em metadados e as soluções existentes para a gestão e armazenamento de documentos fiscais. A partir dessa análise, identificam-se as lacunas que justificam a arquitetura proposta nesta pesquisa.

2.1 *Pipelines* de Ingestão de Dados Adaptativos

A capacidade de se adaptar à evolução de esquemas é um dos principais motivadores para a adoção de arquiteturas de ingestão de dados orientadas por metadados. Chirumamilla (2024) aborda esse problema, ao demonstrar como esquemas abstratos centralizados em repositórios de metadados, permitem modificar modelos de dados, sem reescrever o código do *pipeline* de integração. Enquanto Chirumamilla (2024) aplica essa elasticidade estrutural às mudanças em catálogos de produtos de comércio eletrônico e varejo, nossa pesquisa direciona essa mesma premissa para o cenário da escrituração fiscal.

No Brasil, os arquivos da EFD ICMS IPI possuem leiautes em arquivos texto estruturados em árvore que sofrem revisões governamentais periódicas [6]. O *pipeline* de ingestão aqui proposto inova ao utilizar esquemas JSON como contrato de metadados para interpretar, validar e extrair os registros da EFD ICMS IPI dinamicamente. Dessa forma, quando a legislação fiscal altera a estrutura de um bloco ou adiciona novos campos, a adaptação do sistema requer apenas a atualização do artefato de mapeamento JSON, conferindo alta manutenibilidade ao processamento de dados tributários.

2.2 Armazenamento de documentos fiscais

No contexto da automação fiscal e da modernização dos sistemas de suporte ao ecossistema do SPED, existem pesquisas que focam na descentralização e no ganho de resiliência arquitetural para lidar com o grande volume de obrigações tributárias. Nessa vertente, Tavares (2022) propõem uma arquitetura baseada em microsserviços e contêineres *Docker* para a centralização, baixa e armazenamento de Notas Fiscais Eletrônicas (NF-e), advindas da Secretaria da Fazenda (SEFAZ). O trabalho foca em solucionar os gargalos operacionais da conferência manual e assegurar a guarda dos arquivos XML (Linguagem de Marcação Extendida, tradução livre do inglês *Extensible Markup Language*) em um repositório lógico de armazenamento na nuvem, isolando o consumo e os recursos computacionais de cada empresa, para prover alta disponibilidade e escalabilidade.

Embora Tavares (2022) demonstre a viabilidade de arquiteturas escaláveis para a captura e preservação de documentos fiscais em nível de arquivo físico, a solução limita-se à camada de aquisição e guarda em nuvem de arquivos já bem estruturados em XML. A literatura ainda carece de abordagens robustas voltadas para a ingestão, interpretação e estruturação granular de obrigações acessórias textuais não tão bem estruturadas, como o EFD ICMS IPI.

Nesse sentido, o presente trabalho avança e complementa a linha de investigação de modernização contábil-tributária, ao propor um *pipeline* de ingestão dinâmico e orientado a metadados. Enquanto soluções como a de [8] resolvem o problema do tráfego e do armazenamento bruto do documento, a arquitetura aqui proposta foca no processamento relacional de alta precisão, para assegurar a integridade dos dados em Bancos de Dados Relacionais *PostgreSQL*.

3 Pipeline Baseado em Metadados

Este capítulo detalha os procedimentos metodológicos adotados para o desenvolvimento da solução, abrangendo desde a sua concepção, planejamento e execução, até a contextualização do ambiente no qual o sistema está inserido.

3.1 EFD ICMS IPI e seu Modelo Semiestruturado

O leiaute do arquivo EFD ICMS IPI é formalizado e disponibilizado por meio de Notas Técnicas [6] e do Guia Prático [4] disponibilizados no sítio digital do SPED. Periodicamente eles são atualizados e formalizados, por meio de resoluções e atos normativos da Comissão Técnica Permanente do ICMS [9].

A estrutura atual do EFD ICMS IPI é dividida em blocos [6], os quais subdividem-se em registros, que contêm campos específicos para a inserção dos dados. Em termos gerais, a macroestrutura de um arquivo EFD ICMS IPI obedece à seguinte sequência padronizada:

- **Registro 0000:** Abertura do arquivo e identificação da entidade;
- **Bloco 0:** Abertura, identificação e referências;
- **Blocos B, C, D, E, G, H, K:** Dados e apurações fiscais;
- **Bloco 1:** Informações adicionais e complementares;
- **Bloco 9:** Controle e encerramento do arquivo;
- **Registro 9999:** Encerramento do documento fiscal.

Adicionalmente, cada bloco possui seus próprios registros de abertura e fechamento, estabelecendo uma delimitação clara de escopo (*e.g.*, o Bloco 0 inicia-se no Registro 0001 e finaliza-se no Registro 0990). A organização desses registros ocorre de forma hierárquica, viabilizando aplicação do conceito de “pai e filho” à sintaxe do arquivo. A frequência de aparição de um registro é governada pelas regras de ocorrência (‘1’, ‘V’, ‘1:N’, ‘1:1’). No nível mais granular, os registros são decompostos em campos elementares [6].

Além da sintaxe abstrata, a Nota Técnica [6] estabelece diretrizes físicas essenciais para o processamento computacional: codificação obrigatoriamente em ISO 8859-1 (Latin-1); estrutura de linhas de tamanho variável iniciadas na coluna 1; utilização do caractere *pipe* (|, ASCII 124) como delimitador padrão; e encerramento de linhas demarcado por *Carriage*

Return (CR, ASCII 13) e *Line Feed* (LF, ASCII 10) [6]. Um exemplo desse formato pode ser visto no Quadro 1:

Quadro 1: Exemplo de documento EFD

```
|0000|EMP|0|01012024|31012024|EMPRESA TESTE LTDA
|12345678901234|12345678901|EM|EMPRESA TESTE |EMPRESA|EMPRESA TESTE LTDA
|EMPRESA T|E|1|^CRLF
|0001|0|^CRLF
|0005|EMPRESA TESTE LTDA|EMPRESA |COMERCIAL ABC|EMPRESA TE|FULANO DE TAL|
RUA DAS FLORES 123|EMPRESA TES|COMERCIAL A|COMERCIAL ABC|^CRLF
|0100|COMERCIAL ABC|12345678901|EMPRESA TESTE L|12345678901234|COMERCIA|
contribuinte|COMERCIAL |12345678901234567890|MARIA SANTOS|FULANO DE T|
RUA DAS FLO|FULANO DE TAL|COMERCI|^CRLF
|0990|6|^CRLF
|B001|0|^CRLF
|B020|99|100|SAO PAULO|CO|FU|COM|COMERCIAL|EMPRESA TESTE LTDA|01042024|
FULANO |0,00|1,00|99,99|100,00|999,99|1000,00|1234,11|0,01|999999,99|
PRODUTO A1|^CRLF
|B990|3|^CRLF
|9901|0|^CRLF
|9900|0000|1|^CRLF
|9900|0001|1|^CRLF
|9900|0005|1|^CRLF
|9900|0100|1|^CRLF
|9900|0990|1|^CRLF
|9900|B001|1|^CRLF
|9900|B020|1|^CRLF
|9900|B990|1|^CRLF
|9900|9901|1|^CRLF
|9900|9900|11|^CRLF
|9900|9999|1|^CRLF
|9999|21|^CRLF
```

Fonte: Elaborado pelo autor (2026).

A compreensão desse formato de documento fiscal estabelece o modelo de dados formal sobre o qual se sustenta todo o processo de ingestão de dados deste trabalho.

3.2 Especificação Estruturada e Modelagem de Regras (Metadados)

Devido à ausência de uma definição legível por máquina oficial para a EFD ICMS IPI, a aplicação estrita da especificação técnica dependeria da extração de dados de arquivos em formato PDF (Formato de Documento Portátil, tradução livre do inglês *Portable Document Format*). Um exemplo de Documento Fiscal Eletrônico (DF-e) que não possui esse problema é a Nota Fiscal Eletrônica, que mesmo estando em XML, possui uma especificação em XSD [10] (Definição de *schema* XML, tradução livre do inglês *XML Schema Definition*), permitindo assim validar o formato semiestruturado do XML. A extração das regras em PDF de forma

automatizada é propensa a falhas, visto que o documento original não possui estrutura fixa, gerando entraves para a transcrição do código e para a manutenção contínua das regras.

Para evitar este problema, optou-se por transcrever as diretrizes para o formato JSON de forma manual, sob revisão por pares da equipe de desenvolvimento. Como o sistema consome arquivos EFD ICMS IPI já validados e aprovados pelos programas da SEFAZ, o escopo de implementação dispensou a verificação de regras fiscais. Assim, o mapeamento limitou-se aos requisitos estruturais estritamente necessários para realizar a tradução do formato textual para um formato que represente a estrutura de forma completa dentro do código, bem como a persistência dos dados.

Uma vez garantida a extração e a estruturação correta dos dados em memória, o desafio subsequente consistiu em orquestrar a persistência dessa hierarquia no banco de dados relacional. Para evitar o engessamento do código no componente de carga, foi necessário que o modelo JSON fosse estendido para um contrato de mapeamento abrangente, incorporando ativamente as regras de inserção física, como ilustrado no Quadro 2:

Quadro 2: Exemplo de modelo JSON de regras de metadados para a EFD ICMS IPI

```
{
  "nome_tabela": "cad_apuracao_icms",
  "registro_principal": "e001.e100",
  "is_master": false,
  "registros": [
    {
      "caminho": "",
      "ocorrencia": "n",
      "campos": [
        {
          "nome_campo": "dt_ini",
          "tipo_dado": "date"
        },
        {
          "nome_campo": "dt_fin",
          "tipo_dado": "date"
        }
      ]
    },
    {
      "caminho": "e110",
      "ocorrencia": "1",
      "campos": [
        {
          "nome_campo": "vl_tot_debitos",
          "tipo_dado": "numeric(15, 2)"
        },
        ...
      ]
    }
  ]
}
```

```

...
    {
        "nome_campo": "vl_aj_debitos",
        "tipo_dado": "numeric(15, 2)"
    },
    {
        "nome_campo": "vl_tot_aj_debitos",
        "tipo_dado": "numeric(15, 2)"
    },
    {
        "nome_campo": "vl_estornos_cred",
        "tipo_dado": "numeric(15, 2)"
    },
    {
        "nome_campo": "vl_tot_creditos",
        "tipo_dado": "numeric(15, 2)"
    },
    {
        "nome_campo": "vl_aj_creditos",
        "tipo_dado": "numeric(15, 2)"
    },
    {
        "nome_campo": "vl_tot_aj_creditos",
        "tipo_dado": "numeric(15, 2)"
    },
    {
        "nome_campo": "vl_estornos_deb",
        "tipo_dado": "numeric(15, 2)"
    },
    {
        "nome_campo": "vl_sld_credor_ant",
        "tipo_dado": "numeric(15, 2)"
    },
    {
        "nome_campo": "vl_sld_apurado",
        "tipo_dado": "numeric(15, 2)"
    },
    {
        "nome_campo": "vl_tot_ded",
        "tipo_dado": "numeric(15, 2)"
    },
    {
        "nome_campo": "vl_icms_recolher",
        "tipo_dado": "numeric(15, 2)"
    },
    {
        "nome_campo": "vl_sld_credor_transportar",
        "tipo_dado": "numeric(15, 2)"
    },
    {
        "nome_campo": "deb_esp",
        "tipo_dado": "numeric(15, 2)"
    }
}
],

```

```

"colunas": [
  {
    "nome_coluna": "dt_ini",
    "tipo_dado": "date",
    "origem": "efd",
    "registro": {
      "caminho": "",
      "nome_campo": "dt_ini"
    }
  },
  {
    "nome_coluna": "dt_fin",
    "tipo_dado": "date",
    "origem": "efd",
    "registro": {
      "caminho": "",
      "nome_campo": "dt_fin"
    }
  },
  {
    "nome_coluna": "vl_tot_debitos",
    "tipo_dado": "numeric(15, 2)",
    "origem": "efd",
    "registro": {
      "caminho": "e110",
      "nome_campo": "vl_tot_debitos"
    }
  },
  {
    "nome_coluna": "vl_aj_debitos",
    "tipo_dado": "numeric(15, 2)",
    "origem": "efd",
    "registro": {
      "caminho": "e110",
      "nome_campo": "vl_aj_debitos"
    }
  },
  {
    "nome_coluna": "vl_tot_aj_debitos",
    "tipo_dado": "numeric(15, 2)",
    "origem": "efd",
    "registro": {
      "caminho": "e110",
      "nome_campo": "vl_tot_aj_debitos"
    }
  },
  {
    "nome_coluna": "vl_estornos_cred",
    "tipo_dado": "numeric(15, 2)",
    "origem": "efd",
    "registro": {
      "caminho": "e110",
      "nome_campo": "vl_estornos_cred"
    }
  },
  ...

```



```

...
{
  "nome_coluna": "vl_tot_creditos",
  "tipo_dado": "numeric(15, 2)",
  "origem": "efd",
  "registro": {
    "caminho": "e110",
    "nome_campo": "vl_tot_creditos"
  }
},
{
  "nome_coluna": "vl_aj_creditos",
  "tipo_dado": "numeric(15, 2)",
  "origem": "efd",
  "registro": {
    "caminho": "e110",
    "nome_campo": "vl_aj_creditos"
  }
},
{
  "nome_coluna": "vl_tot_aj_creditos",
  "tipo_dado": "numeric(15, 2)",
  "origem": "efd",
  "registro": {
    "caminho": "e110",
    "nome_campo": "vl_tot_aj_creditos"
  }
},
{
  "nome_coluna": "vl_estornos_deb",
  "tipo_dado": "numeric(15, 2)",
  "origem": "efd",
  "registro": {
    "caminho": "e110",
    "nome_campo": "vl_estornos_deb"
  }
},
{
  "nome_coluna": "vl_sld_credor_ant",
  "tipo_dado": "numeric(15, 2)",
  "origem": "efd",
  "registro": {
    "caminho": "e110",
    "nome_campo": "vl_sld_credor_ant"
  }
},
{
  "nome_coluna": "vl_sld_apurado",
  "tipo_dado": "numeric(15, 2)",
  "origem": "efd",
  "registro": {
    "caminho": "e110",
    "nome_campo": "vl_sld_apurado"
  }
},
...

```

```

...
{
  "nome_coluna": "vl_tot_ded",
  "tipo_dado": "numeric(15, 2)",
  "origem": "efd",
  "registro": {
    "caminho": "e110",
    "nome_campo": "vl_tot_ded"
  }
},
{
  "nome_coluna": "vl_icms_recolher",
  "tipo_dado": "numeric(15, 2)",
  "origem": "efd",
  "registro": {
    "caminho": "e110",
    "nome_campo": "vl_icms_recolher"
  }
},
{
  "nome_coluna": "vl_sld_credor_transportar",
  "tipo_dado": "numeric(15, 2)",
  "origem": "efd",
  "registro": {
    "caminho": "e110",
    "nome_campo": "vl_sld_credor_transportar"
  }
},
{
  "nome_coluna": "deb_esp",
  "tipo_dado": "numeric(15, 2)",
  "origem": "efd",
  "registro": {
    "caminho": "e110",
    "nome_campo": "deb_esp"
  }
},
{
  "nome_coluna": "sq_apuracao_icms",
  "tipo_dado": "serial4",
  "origem": "serial"
},
{
  "nome_coluna": "sqn_efd",
  "tipo_dado": "int4",
  "origem": "registro_pai",
  "tabela_origem": "cad_efd"
}
],
"chave_primaria": [
  "sq_apuracao_icms"
],
"filhos": [
  "cad_icms_ajuste"
]
}

```

Passou-se a definir não apenas só a estrutura interna da EFD ICMS IPI e o relacionamento entre seus registros e as tabelas relacionais do banco de dados, mas também estratégias de desnormalização — isso é, quais registros hierárquicos poderiam ser fundidos em uma única tabela — e inserção — indicando, por exemplo, quais dados compõem determinada coluna de cada tabela.

Essa abordagem otimiza o modelo de dados, mitigando operações de *INNER JOIN* durante a leitura analítica das informações, o que reduz o consumo de E/S e índices ao buscar informações que já são intimamente relacionadas, acelerando consultas ao evitar junções. Além disso, estudar como o *schema* do banco de dados será montado de antemão permite uma maior flexibilidade quanto a estruturação dos metadados. A estrutura final desse contrato, contendo todas as diretrizes requeridas para a realização da carga no estado atual.

Em termos gerais, cada um dos campos cumpre o seguinte papel:

- **nome_tabela:** Representa o nome da tabela no *Datalake* cujo o presente objeto representa;
- **registro_principal:** Representa o registro principal dessa tabela, representado através do caminho real do registro;
- **registros:** Representa uma lista com os registros cujas informações são armazenadas na presente tabela;
 - **caminho:** Representa o nome do presente registro, representado através do caminho relativo ao *registro_principal*. Se vazio, indica que se trata do registro principal da presente tabela;
 - **ocorrencia:** Representa o número de ocorrências do presente registro em relação ao pai dele;
 - **campos:** Representa a lista de campos do registro atual, através de seus nomes (*nome_campo*) e dos tipos de dados esperados (*tipo_dado*);
- **colunas:** Representa uma lista com as colunas que fazem parte da presente tabela;
 - **nome_coluna:** Representa o nome da presente coluna;
 - **tipo_dado:** Representa o tipo de dado armazenado na presente coluna;
 - **origem:** Representa a origem dos dados armazenados na presente coluna. Pode ser um dos seguintes:
 - * **efd:** Representa que os dados armazenados na presente coluna derivam diretamente da subárvore atual da EFD ICMS IPI;
 - * **metadados:** Representa que os dados armazenados na presente coluna derivam dos metadados do arquivo EFD ICMS IPI;

- * **serial:** Representa que os dados armazenados na presente coluna derivam de sequencial interno ao banco de dados;
 - * **externo:** Representa que os dados armazenados na presente coluna derivam de uma consulta que devem ser realizado em outra tabela do banco de dados;
 - * **registro_pai:** Representa que os dados armazenados na presente coluna derivam da chave primária de um registro pai.
- **extras:** Representa informações adicionais que possam ser necessárias para obter os dados armazenados na presente coluna. Por exemplo, se a **origem é efd**, teremos dois campos adicionais, um que indica qual o caminho que deve ser seguido para encontrar aquele o campo que contém a informação, e o nome desse campo;
- **chave_primaria:** Representa a lista de colunas que compõe a chave primária da tabela.

3.3 Arquitetura da Solução de Ingestão de Dados

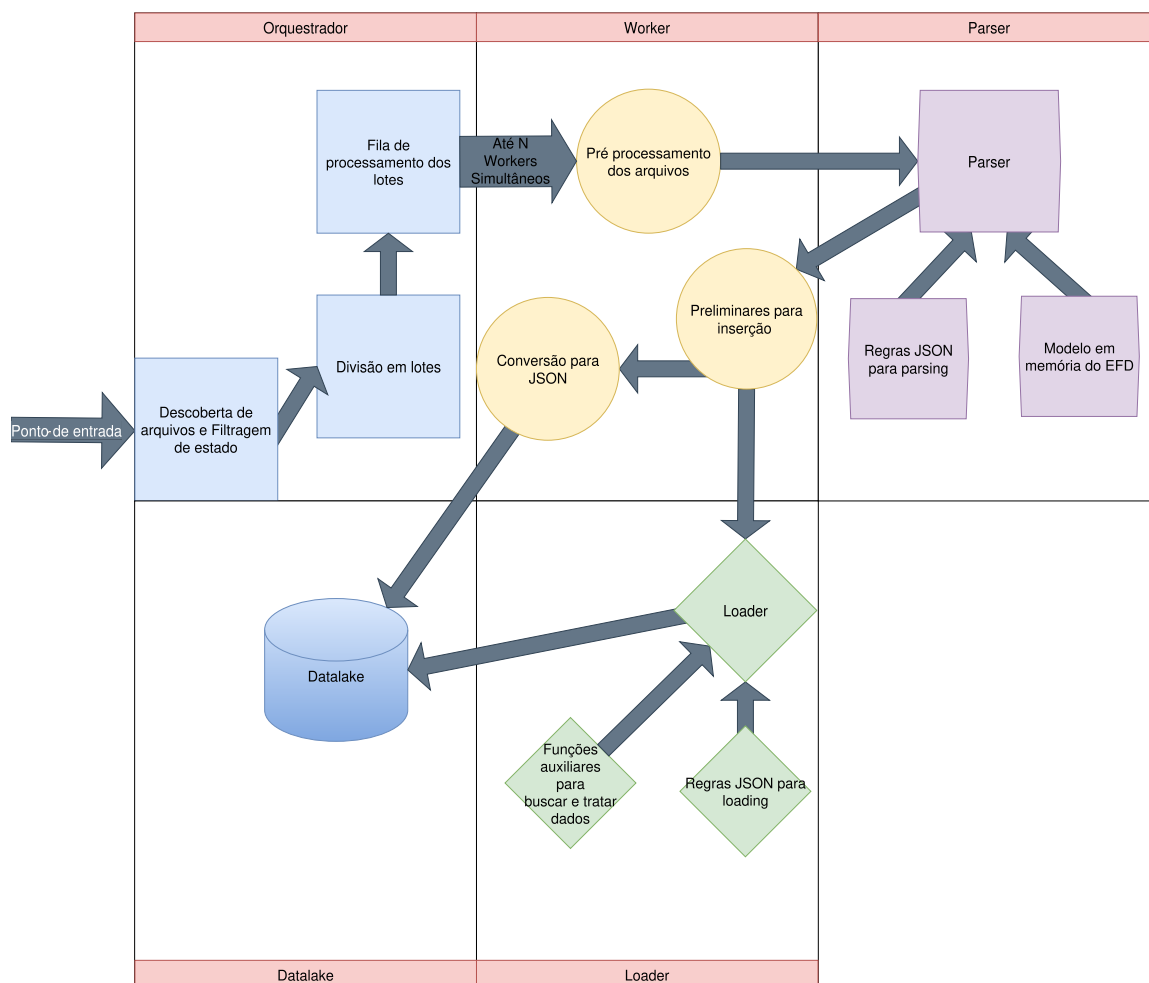
A finalidade do projeto consiste no desenvolvimento de uma aplicação capaz de processar os arquivos brutos da EFD ICMS IPI, transformá-los em uma representação intermediária em memória e persisti-los em um banco de dados *PostgreSQL*, tanto em formato JSON — via *JSON Binary*, ou apenas JSONB —, quanto em esquema relacional normalizado. Vale ressaltar aqui que o foco do *pipeline* não é a ingestão em tempo real de documentos EFD ICMS IPI, dado que tais arquivos são gerados mensalmente [11], o que gera um espaço amplo para a ingestão dos mesmos antes das auditorias serem possíveis. A arquitetura geral do processo de ingestão projetado para este fim pode ser visualizada na Figura 1, e as quatro componentes arquiteturais principais serão detalhadas a seguir.

3.3.1 Orquestrador

O Orquestrador atua como o ponto de entrada do *pipeline*. Sua função não é manipular os dados diretamente, mas sim coordenar o fluxo de trabalho, desde a localização dos arquivos, até a distribuição de tarefas. Acionado via interface de linha de comando, ele valida as regras JSON, diretórios e conexões, executando um ciclo de vida em cinco etapas operacionais:

1. **Descoberta:** Varredura de diretórios para identificação de arquivos aderentes ao padrão esperado.
2. **Filtragem de Estado:** Consulta ao *log* de execuções anteriores para isolar e remover arquivos já inseridos com sucesso.

Figura 1: Diagrama de arquitetura da solução de ingestão para arquivos EFD ICMS IPI.



Fonte: Elaborado pelo autor (2026).

3. **Divisão em Lotes (*Batches*):** Agrupamento dos arquivos pendentes em lotes de tamanho fixo, balanceando a carga de processamento.
4. **Fila de processamento dos lotes via *Workers*:** Os lotes criados na etapa anterior são atribuídos a *Workers* paralelos e independentes, que concorrem apenas pelo acesso ao banco e, caso falhem, não afetarão os demais. Esses *Workers* são criados sobre demanda e, para melhor uso dos recursos computacionais, o número de instâncias ativas ao mesmo tempo é controlado por semáforos¹, sendo limitado a um máximo pré-estabelecido ao início da execução do *pipeline*. Normalmente esse é um valor igual ou menor ao número de núcleos de processamento disponíveis no momento.
5. **Finalização:** Geração de relatórios de estatísticas de execução do *pipeline* e garantia de encerramento seguro para evitar vazamento de memória e conexões retidas.

¹Mecanismo de sincronização utilizado em programação concorrente para limitar a quantidade de processos ou *threads* que acessam simultaneamente um recurso.

3.3.2 *Worker*

O *Worker* é a unidade de processamento independente, que executa o *pipeline* fim a fim para um lote específico. Em sua inicialização, estabelece uma conexão dedicada com o banco de dados. A partir disso, ele executa a ingestão de forma individual para cada arquivo, garantindo o isolamento de falhas. Ele primeiramente realiza a sanitização do arquivo (verificando corrupção ou ausência de dados) e depois submete o texto ao componente *Parser*.

Após obter a representação em árvore gerado pelo componente *Parser*, o *Worker* inicia uma transação atômica no banco de dados para realizar a carga normalizada e a inserção no formato JSON. Estratégias de tratamento de erro segmentam as falhas operacionais: exceções a nível de arquivo são ignoradas e registradas sem interromper o lote, enquanto quedas de conexão forçam tentativas de reconexão antes de avançar o cursor.

3.3.3 *Parser*

Este módulo representa o primeiro estágio do *pipeline* de ingestão que lida diretamente com o formato de dados do ecossistema EFD ICMS IPI. Ele é responsável pela conversão do formato textual dos arquivos para uma representação intermediária em memória, organizando os registros fiscais em uma árvore de objetos categorizada por grupos (0, 1, B, C, D, E, G, H). Esta representação é responsável por alimentar todos os modos de persistência disponíveis no sistema.

O processamento adota uma estratégia de varredura sequencial, iterando linha a linha sobre o arquivo passado como argumento. A cada linha não vazia, o algoritmo particiona o conteúdo pelo delimitador *Pipe* ('|'), identifica o código do registro por meio do primeiro campo — como visto no Quadro 1 — e, em seguida, consulta o JSON contendo os metadados para encontrar o seu caminho hierárquico dentro da estrutura, a sua cardinalidade de ocorrência e quais os campos esperados nesse registro.

O mecanismo de inserção na estrutura em memória opera por descida recursiva para varrer o caminho hierárquico: para cada registro, consulta-se o percurso predefinido em seus metadados e, a cada segmento do caminho, cria-se um nó intermediário, caso este ainda não exista. A criação desses nós respeita a cardinalidade do registro ancestral — associações do tipo um-para-um, para pais de ocorrência única, ou um-para-vários, para pais de ocorrência múltipla, utilizando uma lista para acumular as ocorrências. Ao final da travessia, o item é inserido no nó terminal, conforme sua própria cardinalidade: registros que devem aparecer uma única vez são inseridos como atributo direto, enquanto registros de ocorrência múltipla são adicionados a uma lista.

Aqui também são feitos alguns tratamentos de erro que ajudam a assegurar a qualidade do processamento das EFD ICMS IPI:

- Caso um registro de cardinalidade unitária apareça mais de uma vez no mesmo contexto, o processamento é interrompido com uma sinalização de erro.
- Há também uma tolerância com variações entre o número de campos encontrados e os extraídos:
 - Ela se deve ao fato que versões antigas do leiaute fiscal costumam conter menos campos em alguns registros, já que o leiaute tende a evoluir para acomodar mais informações ao longo do tempo.
 - Portanto, registro com menos campos que o esperado tem um preenchimento com valores vazios para campos faltantes.
 - Contudo, se o registro possuir mais campos que o esperado, o processamento é abortado com indicação de inconformidade.

Esse tratamento de erros segue uma política de interrupção imediata: qualquer falha — seja de cardinalidade, de número de campos, de grupo não mapeado, de ausência do registro terminal que marca o fim do arquivo ou falha interna — é registrada em *log* e propagada como exceção para o *Worker*, interrompendo o processamento do arquivo. Essa decisão de projeto privilegia a consistência dos dados em detrimento da tolerância a falhas, evitando que um arquivo malformado gere cargas parciais ou corrompidas no banco de dados relacional. Portanto, este módulo estabelece uma separação clara entre a lógica de análise sintática e a de inserção no banco de dados — permitindo um pré-processamento do arquivo antes da carga ser realizada. Essa separação confere ao *pipeline* alta coesão funcional e baixo acoplamento, o que melhora a manutenibilidade do sistema.

3.3.4 *Loader*

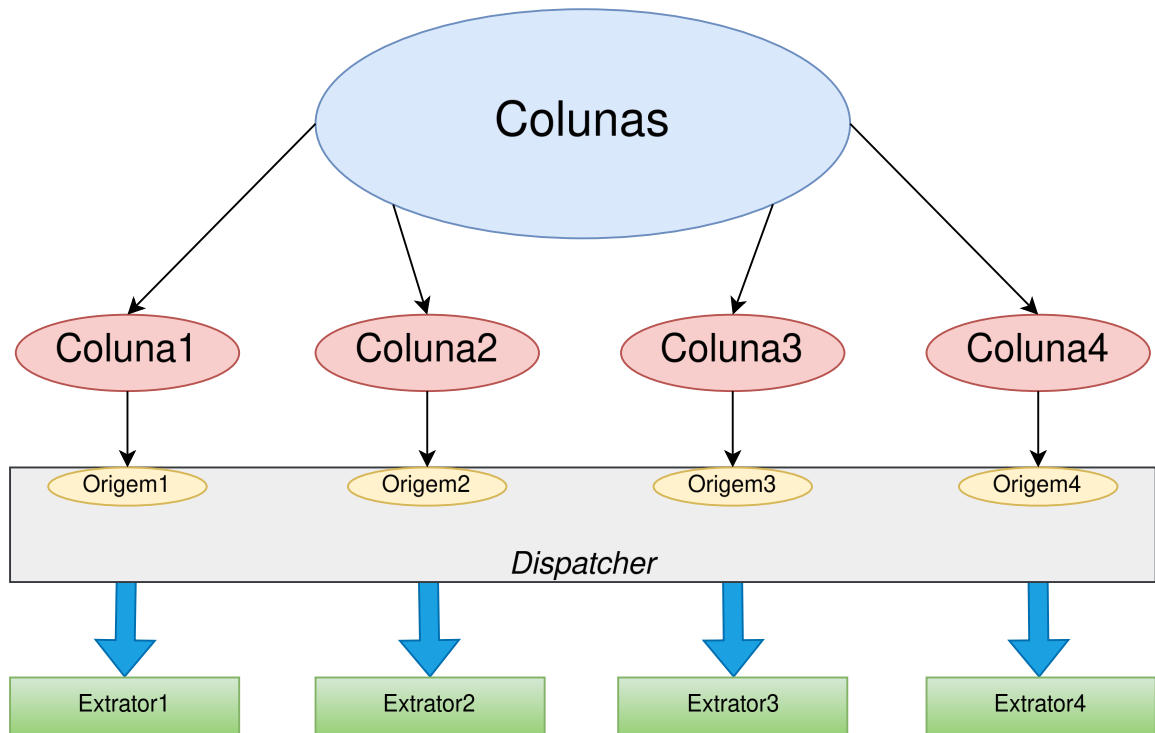
Este módulo representa o segundo estágio da ingestão de dados do sistema, sendo responsável por orquestrar a inserção do formato hierárquico gerado no item anterior no banco de dados *PostgreSQL*. A arquitetura aqui possui, além da dependência do JSON de metadados, que possui as informações utilizadas para inserção, também há a dependência de um módulo interno para extração dos dados para cada coluna de cada tabela presente nos metadados, que representa a nível de implementação a execução da *origem* dos dados apresentado na Seção 3.2.

Esse módulo utiliza de *pattern matching*² sobre o campo *origem* de cada item do campo

²Mecanismo de programação que verifica a correspondência de uma estrutura ou valor de dados com um padrão predefinido, acionando fluxos de código específicos sem a necessidade de estruturas condicionais explícitas.

colunas do JSON de metadados, como visto no Quadro 2. Esse *pattern matching* funciona como despachante para determinar qual a função de extração deve ser usada para preencher a coluna representada pelo item atual, como mostra a Figura 2.

Figura 2: Diagrama exemplificando a lógica do despachante.



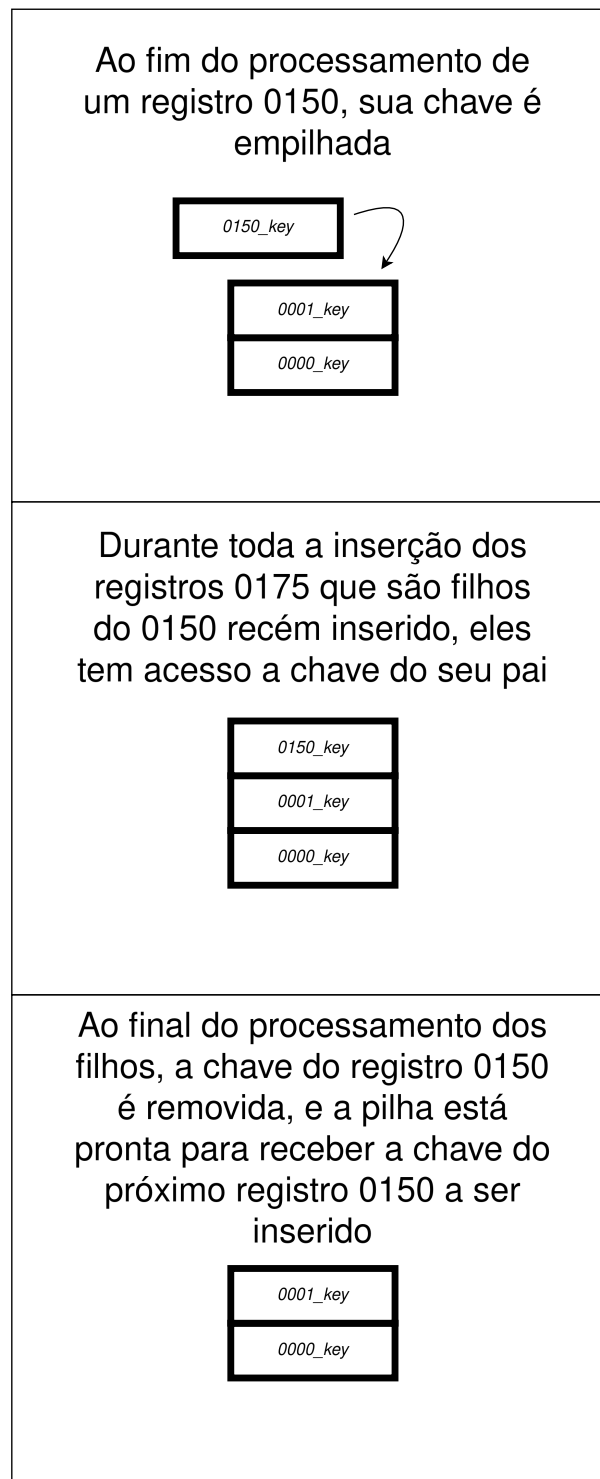
Fonte: Elaborado pelo autor (2026).

Ou seja, a origem declarada nos metadados da coluna determina em tempo de execução qual algoritmo de extração será aplicado, sem que seja necessário um dispêndio condicional explícito para cada coluna.

Em relação ao processamento, ele começa com o registro 0000, que é a raiz do EFD ICMS IPI. A partir daqui, os registros são varridos via um algoritmo de descida recursiva de busca em profundidade (*Depth-First Search*, DFS).

Para dispersão das chaves primárias de cada pai para repasse aos seus filhos, o *Loader* mantém uma pilha implícita, cujo o funcionamento no caso é exemplificado na Figura 3.

Figura 3: Diagrama exemplificando a lógica de dispersão das chaves primárias dos pais.



Fonte: Elaborado pelo autor (2026).

Esse padrão assegura que, ao final da execução, todos os registros tenham sido inseridos na ordem correta, respeitando as dependências referenciais entre pais e filhos.

Por fim, é válido mencionar que o tratamento de erros nesse módulo também segue a política de consistência dos dados em detrimento da tolerância à falha. Diversos erros são catalogados nessa fase, como: Ausência do registro 0000; Falta das informações necessárias para realização da extração dos dados; Uma das extrações de dados acaba falhando; etc. Cada um desses erros resulta numa falha de inserção que realiza *rollback*³ em todo o processo, evitando assim a existência de dados corrompidos na base.

³Mecanismo de controle de transações em banco de dados que reverte todas as operações executadas desde o início do bloco atual em caso de erro, restaurando a base ao seu estado anterior íntegro.

4 APRESENTAÇÃO E ANÁLISE DOS RESULTADOS

Este capítulo detalha os procedimentos usados para testar, validar e avaliar a qualidade do processo desenvolvido, contemplando desde os pontos fortes da abordagem, erros encontrados e o que pode ser melhorado.

4.1 Ambiente de execução e hardware utilizado

Para a avaliação de desempenho, bem como a execução do *pipeline* de processamento proposto neste trabalho, foi utilizado um ambiente virtualizado disponibilizado pela SEFAZ. A documentação desse ambiente tem por objetivo garantir reprodutibilidade do que será apresentado e fornecer uma base para análises futuras, especialmente no que tange ao paralelismo central da aplicação.

Todos os processos do sistema foram executados em uma única máquina virtual hospedada sob um *hypervisor* *VMWare* com virtualização total. As especificações dos recursos alocados para esta instância encontram-se no Quadro 3.

Quadro 3: Especificações do ambiente computacional

Componente	Especificação Técnica
Infraestrutura	Máquina Virtual sob <i>hypervisor</i> VMware (<i>virtualização total</i>)
Processador	Intel® Xeon® Gold 5320 @ 2.20GHz
Topologia da CPU	16 núcleos lógicos (16 <i>sockets</i> , 1 <i>core/socket</i> , 1 <i>thread/core</i>)
Cache	L1: 80 KB / L2: 1,25 MB / L3: 39,9 MB (Compartilhado)
Memória RAM	93Gi, com 4Gi de swap
Armazenamento (Raiz do sistema)	SSD de 135GB
Armazenamento (Volume dedicado ao banco de dados)	SSD de 36TB
Versão do PostgreSQL	16

Fonte: Elaborado pelo autor (2026).

A infraestrutura provisionada apresenta características altamente favoráveis para o perfil de carga de trabalho do sistema desenvolvido. Destaca-se principalmente a topologia de processamento disponibilizada: a alocação de 16 processadores lógicos concentrados em um único nó NUMA, garantindo um tempo de acesso uniforme à memória RAM. Na prática, isso permitiu

com que o Orquestrador instanciasse até 16 *Workers*, saturando a capacidade de processamento paralelo, sem sofrer degradação devido a penalidades de troca de contexto. O banco de dados utilizado para testes foi instanciado na mesma máquina virtual, o que manteve os testes isolados de eventuais flutuações relacionadas à latência de rede.

4.2 Metodologia de testes

A validação do *pipeline* de ingestão desenvolvido foi conduzido por meio testes experimentais de caráter preliminar, em grande parte de natureza *ad-hoc*. Devido a restrições de tempo, experiência da equipe e escopo do projeto, não foi possível a realização de uma bateria de testes ou de uma coleta de dados em situação de estresse em ambiente de produção real.

Contudo, cumpre destacar que os cenários selecionados para os testes *ad-hoc* foram projetados estrategicamente para cobrir fluxos críticos de execução do sistema, servindo como uma validação da arquitetura proposta. Para tanto, a massa de dados de teste consistiu em 100 arquivos EFD ICMS IPI reais, número este que foi escolhido pela equipe por ser considerado um número representativo, mas não houve nenhuma avaliação da eficácia real dessa escolha. Também foram usados diversos arquivos EFD ICMS IPI gerados artificialmente, buscando cobrir as seguintes especificações:

- Arquivos contendo uma unidade de cada um dos registros mapeados;
- Arquivos com ausência de registros que foram normalizados;
- Arquivos cobrindo os diversos casos de um-para-vários;
- Arquivos com registros contendo menos campos que o total mapeado;
- Arquivos contendo casos de um-para-vários onde se espera um-para-um;
- Arquivos sem a presença do registro 0000;
- Arquivos com campos que quebram as regras de formatação especificada;
- Arquivos com campos que não quebram as regras de formatação, mas não conseguem ser inseridos por limitações técnicas. Ex: Campos numéricos que causam *overflow*;
- Arquivos com registros contendo mais campos que o total esperado;

Esta composição visou cobrir possíveis lacunas e anomalias que os documentos reais pudessem deixar de apresentar.

Uma das estratégias adotadas para maximizar a eficácia dos testes *ad-hoc* concentrou-se na verificação da integridade absoluta da informação trafegada ao longo do *pipeline*. Empregou-se uma idéia semelhante a técnica de *round-trip validation*, que consiste na validação dos dados mediante a reconstrução reversa da informação original a partir dos registros já processados. Apesar da equipe não construir um processo totalmente automatizado e rico como o esperado pelo método *round-trip validation*, ainda foi possível verificar a premissa central de que, se o sistema é capaz de desestruturar, transformar e persistir os dados corretamente, deve ser igualmente capaz de remontá-los em seu formato bruto original, sem qualquer depreciação. O procedimento foi dividido em duas etapas de verificação isoladas, valendo-se de análises diferenciais de texto:

1. **Validação em Memória (*Parser*):** O arquivo EFD ICMS IPI original foi processado em memória e convertido para o formato estruturado em JSON. Em seguida, aplicou-se uma rotina de engenharia reversa para reconverter esse JSON de volta para o formato de texto plano original. Além disso, o arquivo EFD ICMS IPI original foi normalizado para evitar divergências devido à representações equivalentes, como costuma ocorrer entre campos numéricos. A comparação entre o arquivo de origem e o gerado a partir do JSON atestou a precisão da modelagem estrutural em árvore;
2. **Validação de Persistência (*Loader*):** Para homologar a carga no banco de dados, o ciclo de ingestão foi concluído com a inserção dos registros no *PostgreSQL*. Posteriormente, utilizando exclusivamente as chaves primárias para rastrear os relacionamentos hierárquicos entre as tabelas, os registros foram consultados, extraídos e remontados de volta, no formato original, seguindo o mesmo processo do caso anterior a partir daqui.

Em ambos os cenários de teste, a verificação diferencial linha a linha confirmou a exatidão do processo. A ausência de discrepâncias relevantes provou empiricamente que o sistema cumpre o requisito de extração e transformação, sem incorrer em perdas de informação, truncamento de valores, ou corrupção na hierarquia de dependência dos registros fiscais.

Outra abordagem consistiu na elaboração de documentos contendo as falhas de construção pré-determinadas listadas anteriormente, inserindo-os nos lotes de processamento para avaliação de comportamento. Este ensaio buscou aferir a resiliência do sistema no tratamento de arquivos defeituosos e avaliar a capacidade de rastreabilidade dos erros, permitindo diagnosticar se a falha advém do arquivo, de equívocos nos metadados ou da lógica de código.

Neste cenário, os resultados mostraram-se satisfatórios. Os *logs* gerados apresentaram um nível adequado de detalhamento (Listagem 4). A identificação explícita do erro, associada ao nome do arquivo, viabiliza a averiguação direta da anomalia. Adicionalmente, constatou-se que a ocorrência não comprometeu o *pipeline*, o qual prosseguiu sua execução, sem sofrer

interrupções globais. Este comportamento atesta empiricamente que o sistema isola arquivos defeituosos, otimizando a utilização dos recursos alocados para o respectivo lote.

Quadro 4: Exemplo de log de erro de *parsing*

```
12:11:04,445 INFO ERROR processing EFDs/F00000005_AAA+AAAA_EFD_999-9999-MOCK
-9999999999999999-999999999-0-202512-17122025144834-999-17122025144834.txt
.zip: Error: 0005 marked as single occurrence per parent, but more than
one appeared:
Register:|0005|EMPRESA TESTE LTDA|EMPRESA |COMERCIAL ABC|EMPRESA TE|
FULANO DE TAL|RUA DAS FLORES 123|EMPRESA TES|COMERCIAL A|COMERCIAL ABC|
```

Fonte: Elaborado pelo autor (2026).

A estratégia final compreendeu a injeção de erros artificiais durante a execução do *pipeline*, especificamente durante a execução do componente *Loader*. Os erros considerados foram:

- Falha de conexão com o *PostgreSQL*;
- Inconformidade entre o dado a ser inserido e o tipo esperado;
- Datas inválidas.

Tais erros foram atrelados apenas ao processamento de determinados arquivos, com o objetivo de validar a atomicidade da carga diante de falhas no meio do processo e verificar a precisão do registro do erro.

Neste caso, os resultados também foram positivos. Conforme os *logs* (Listing 5), a nomeação padronizada da falha e do arquivo proporcionou boa rastreabilidade. Validou-se que todos os demais arquivos do lote atravessaram o fluxo de carga com sucesso, enquanto o arquivo defeituoso não produziu qualquer efeito no banco de dados. Isso comprova que o sistema mantém a atomicidade das operações, revertendo inserções parciais por meio de *rollback* quando anomalias interrompem o fluxo.

Quadro 5: Exemplo de log de erro durante a carga

```
DETAIL: A field with precision 15, scale 2 must round to an absolute value
less than 10^13.
13:30:53,49 INFO ERROR processing EFDs/F00000049_AAA+AAAA_EFD_999-9999-MOCK
-9999999999999999-999999999-0-202512-17122025144834-999-17122025144834.txt
.zip: DB insert error for b020: numeric field overflow
```

Fonte: Elaborado pelo autor (2026).

Portanto, conclui-se que a avaliação do sistema atendeu com êxito a três critérios fundamentais:

1. **Consistência do componente *Parser*:** Verificação de que a árvore de registros gerada em memória refletiu com fidelidade a hierarquia descrita na especificação fiscal, abrangendo as particularidades estruturais e processando todos os dados;
2. **Aderência aos Metadados:** Validação da capacidade de interpretar dinamicamente o arquivo JSON de regras, assegurando sua aplicação, isolando arquivos não conformes e alternando entre os modos de inserção adequados;
3. **Atomicidade da Carga:** Confirmação de que o componente *Loader* realizou a persistência normalizada de maneira íntegra, respeitando as restrições de integridade referencial e executando *rollback* imediato em caso de falhas, protegendo o SGBD contra dados corrompidos.

Embora os dados coletados não permitam traçar curvas generalizáveis de escalabilidade linear sob concorrência extrema de *Workers*, os resultados obtidos atestam a viabilidade técnica da solução e a correteza lógica de todos os módulos da arquitetura.

4.3 Limitações encontradas após a modelagem

As subseções a seguir detalham gargalos e desafios arquiteturais identificados após a modelagem e durante as etapas de execução do *pipeline*.

4.3.1 Limitações do Tipo JSONB com Grandes Volumes

Após a conclusão dos ensaios preliminares, o sistema foi avaliado em um ambiente de homologação controlado, visando observar seu comportamento perante um volume de dados maior e mais condizente com cenários reais. Um dos imprevistos ocorridos esteve diretamente relacionado às restrições físicas de armazenamento do SGBD *PostgreSQL* no tratamento de arquivos EFD ICMS IPI de grande escala.

O projeto original previa que, em paralelo à carga relacional normalizada, o *Worker* realizaria a conversão do arquivo completo para o formato JSON bruto (*raw data*) para persistência em uma única coluna do tipo JSONB. Contudo, ao submeter arquivos de volumetria massiva (centenas de milhares de registros), o SGBD disparou erros de estouro de limite físico, interrompendo as transações. Esses arquivos chegaram a ocasionar falhas no próprio *pool*⁴ de

⁴Conjunto de conexões ativas e reutilizáveis mantido em memória pelo sistema, evitando o custo computacional de abrir e fechar novas conexões com o banco a cada operação.

conexões com o banco, resultando no cancelamento da inserção de lotes inteiros, mesmo com a configuração de *retries*⁵ habilitada.

Essa falha decorre dos limites estáticos impostos pelo *PostgreSQL* para a alocação de objetos JSONB. O erro mais recorrente foi a violação do limite de 256 MB para campos únicos, embora também tenham sido registradas instâncias de estouro do limite máximo de 1 GB estipulado pelo SGBD.

Para mitigar essa limitação, determinou-se que a inserção de arquivos brutos monolíticos deve ser evitada em casos de alta volumetria. Incorporou-se uma cláusula de verificação que estima o tamanho do JSON gerado antes da operação de E/S — Sigla para *Entrada e Saída*, que representa operações que acessam recursos externos ao programa, como o SGBD. Caso a estimativa ultrapasse o limite configurado, a persistência no campo JSONB é suprimida, garantindo que o processamento normalizado avance sem interrupções.

A descoberta deste gargalo evidenciou que a persistência de superobjetos estruturados em SGBDs relacionais representa um anti-padrão de engenharia para lidar com volumes característicos de *Big Data*⁶, gerando conhecimento valioso para a evolução arquitetural da ferramenta e do modelo de dados utilizado pela equipe.

4.3.2 Degradação da inserção

Durante a validação do componente *Loader* com arquivos EFD ICMS IPI massivos, observou-se uma degradação não linear no tempo total de persistência. Embora a ingestão de dados em tempo real não figure entre os requisitos não funcionais deste trabalho, a análise dessa latência apontou para um gargalo intrínseco à sistemas de ingestão de dados massivos.

A primeira hipótese para a raiz do comportamento foi a dependência estrutural de chaves primárias artificiais geradas sequencialmente pelo *PostgreSQL*. Devido à natureza hierárquica da obrigação fiscal, a persistência exige a inserção prévia do registro-pai para obtenção de seu identificador único gerado pelo banco. Somente após receber este valor, o componente *Loader* é capaz de propagá-lo como chave estrangeira aos registros-filhos.

Contudo, as investigações demonstraram que a degradação de desempenho observada não decorre unicamente dessa dependência. Uma remodelagem experimental eliminando a necessidade de consultas prévias de identificadores foi implementada e testada, porém o comportamento de perda progressiva de vazão persistiu de forma análoga.

⁵Estratégia de resiliência que agenda a reexecução automática da criação de uma conexão com o banco de dados antes de declará-lo definitivamente fora de alcance.

⁶Termo que descreve conjuntos de dados massivos, complexos e de rápido crescimento, cuja escala e variedade ultrapassam a capacidade de armazenamento, gestão e processamento de SGBDs relacionais convencionais.

A análise empírica refinada nos levou a concluir que a verdadeira causa raiz reside na sobrecarga imposta ao motor de persistência do *PostgreSQL*, durante a inserção de volumes massivos sob um único bloco transacional. À medida que milhões de registros hierárquicos são injetados continuamente, o banco de dados é forçado a manter em tempo real a consistência das restrições referenciais e a integridade de múltiplos índices estruturados. Esse processo de atualização dinâmica satura os recursos de E/S e degrada o tempo de execução de forma não-linear, conforme o volume do documento cresce.

A desativação momentânea das restrições de consistência para a realização da carga atestou o ganho de desempenho, diminuindo consideravelmente o tempo necessário para a execução total da carga. Contudo, a resolução definitiva deste gargalo não pode contemplar apenas essa escolha, e exige estratégia de ingestão de alta complexidade, envolvendo uma modelagem dedicada do funcionamento do *PostgreSQL* para ingestão massiva de dados em único bloco transacional. Diante dessa complexidade, e do fato de que a abordagem atual atende satisfatoriamente aos requisitos de negócio, para a grande maioria dos cenários, a implementação de um novo modelo de persistência otimizado foi deliberadamente isolada do escopo atual, configurando-se como uma oportunidade promissora para trabalhos futuros.

4.3.3 Documentação conflitante

Durante as homologações, observou-se uma ocorrência errônea em relação à base normativa. O registro H020, hierarquicamente subordinado ao registro H010, encontra-se categorizado com multiplicidade um-para-um no Guia Prático [4] oficial da EFD. Não obstante, uma parcela ínfima dos arquivos analisados possuía múltiplos registros H020 para um mesmo H010.

Após averiguação, constatou-se que a Nota Técnica [6] correspondente indicava tal relacionamento como um-para-vários. Para mitigar o impasse normativo e garantir maior flexibilidade à extração, assumiu-se a especificidade da Nota Técnica [6] como a diretriz válida para os metadados do sistema. Este evento evidencia que as especificações oficiais emitidas pelo fisco são suscetíveis a divergências internas, exigindo processos contínuos de governança de dados e adaptação dinâmica das regras de negócio.

4.4 Métricas Operacionais Preliminares e Linha de Base

Embora o escopo central deste trabalho resida na validação da arquitetura proposta quanto a consistência dos dados inseridos e do uso adequado das regras em JSON, o monitoramento das execuções no ambiente de homologação controlado permitiu a coleta empírica de métricas operacionais que estabelecem uma linha de base para trabalhos futuros. Os ensaios registraram

o tempo bruto de execução do *pipeline* em relação ao volume total de arquivos submetidos, englobando as fases de orquestração, transformação estrutural e carga no banco de dados.

A Tabela 1 consolida os resultados quantitativos extraídos do ambiente de homologação, considerando um Orquestrador trabalhando com 8 *Workers* simultâneos. Esse valor foi escolhido por consumir cerca da metade do poder de processamento do ambiente disponibilizado, garantindo que flutuações de desempenho causadas por outras atividades sendo executadas fosse minimizado. A Tabela 2 reúne resultados estatísticos acerca dos *Workers* executados, cada um imbuído de processar 100 arquivos EFD ICMS. O número 100 foi escolhido ao acaso e, apesar de constituir um vetor de exploração interessante para a métrica de desempenho, não foram realizados testes com outros valores.

Tabela 1: Tempo bruto de execução por volumetria

Número da instância de execução	Número de EFDs	de Volume Aproximado	Tempo Total de Execução
1	194668 arquivos	9.6G	21h 36m 25s
2	200040 arquivos	11G	28h 09m 31s
3	242653 arquivos	11G	21h 31m 16s

Fonte: Elaborado pelo autor (2026).

Tabela 2: Estatísticas por lote (*Baseline*)

Número da instância de execução	<i>Worker</i> mais rápido	<i>Worker</i> mais lento	Tempo médio	Desvio padrão
1	0,5m	56,9m	5,3m	5,0m
2	0,5m	76,8m	6,7m	6,0m
3	0,7m	119,6m	8,7m	11,7m

Fonte: Elaborado pelo autor (2026).

Destaca-se que não foram aplicados comparativos diretos com ferramentas comerciais de mercado. Isso se deve ao fato da literatura carecer de referenciais abertos semelhantes ao explorado no presente trabalho.

Ademais, o monitoramento ativo dos lotes forneceu um panorama sobre a integridade da base de dados fornecida. De um total de 637.361 arquivos submetidos ao *pipeline*, o sistema diagnosticou e isolou 15 arquivos contendo anomalias estruturais ou normativas (aproximadamente 0,002% da amostra), dentre essas nenhuma envolvendo a especificação em JSON criada.

Este índice quantifica a suscetibilidade a erros inerente às obrigações fiscais e justifica, na prática, o esforço arquitetural investido nos mecanismos de resiliência, isolamento de falhas e rastreabilidade discutidos nas seções anteriores.

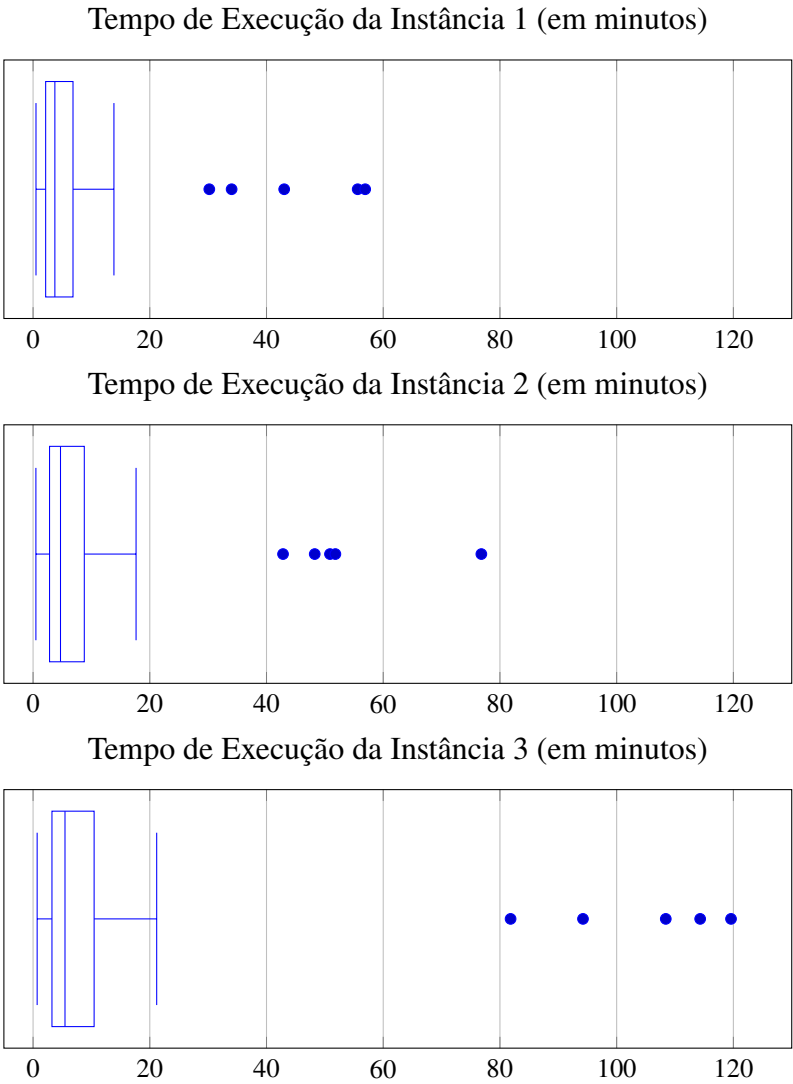
Junto a isso, a alta variação entre o tempo de execução de cada *Worker*, chegando a duração de 119 minutos no pior caso total, exemplifica o que foi exibido na Seção 4.3.2, onde foi comentado como determinados *Workers* sofrem degradação no tempo de execução devido a arquivos massivos. Ainda assim, é válido ressaltar que ao longo dos três lotes executados, cerca de 90% dos *Workers* (88% nos lotes 1 e 2, 93% no lote 3) permaneceu a menos de um desvio padrão em torno da média aritmética, *i.e.*, cerca de 90% dos *Workers* demora não mais que 20 minutos para executar todo seu *pipeline*, o que denota uma boa estabilidade operacional, salvo os casos onde foi evidenciado a degradação do tempo de execução. O *boxplot* exibido na Figura 4 permite uma visualização melhor do comportamento do tempo de execução do *pipeline* em relação ao tempo de execução de cada *Worker*, destacando em cada um deles, os 5 *Workers* com maior tempo de execução.

4.5 Síntese dos Resultados

Os ensaios experimentais conduzidos demonstraram que a adoção de uma arquitetura orientada a metadados e estruturada em arquivos JSON é tecnicamente viável e altamente íntegro para o processamento de obrigações fiscais complexas, como a EFD ICMS IPI. A bateria de testes, mesmo com sua implementação embrionária, atestou a corretude e a precisão do algoritmo de conversão, provando a ausência de perdas informacionais durante o fluxo de desestruturação e carga.

Simultaneamente, a implementação revelou gargalos inerentes à modelagem relacional de grandes volumes, notadamente as limitações de paginação do tipo JSONB e a latência de operações de E/S em inserções hierárquicas massivas. Tais descobertas, no entanto, não invalidam a ferramenta, pelo contrário, balizaram decisões arquiteturais de mitigação e evidenciaram que o sistema lida com anomalias físicas e lógicas de forma isolada, resiliente e rastreável. De modo geral, o *pipeline* cumpriu o propósito de transformar arquivos textuais não estruturados em uma base de dados relacional íntegra, estabelecendo uma fundação sólida para a construção de sistemas de auditoria fiscais.

Figura 4: Distribuição dos tempos de processamento dos *Workers* em minutos em cada instância de execução.



Fonte: Elaborado pelo autor (2026).

5 CONCLUSÕES E TRABALHOS FUTUROS

O presente trabalho teve como objetivo central conceber e validar uma arquitetura de *pipeline* de ingestão de dados orientada a metadados, voltada para a extração, transformação e carga de arquivos fiscais, especificamente a EFD ICMS IPI. A motivação para o desenvolvimento desta solução fundamentou-se na rigidez dos sistemas tradicionais, que frequentemente exigem refatoração de código-fonte a cada nova atualização normativa do fisco.

Os resultados obtidos ao longo das etapas de homologação confirmam que a hipótese arquitetural proposta é tecnicamente viável e altamente vantajosa. A adoção de um modelo de metadados, baseado em arquivos JSON, permitiu desacoplar as regras de negócio da lógica de extração, apesar da necessidade de extração manual dessas regras de negócios a partir dos documentos oficiais.

Adicionalmente, as técnicas de validação utilizadas, mesmo que ainda embrionárias, atestaram a integridade do processo. A precisão verificada na remontagem dos arquivos brutos, a partir dos dados em memória e do banco de dados relacional, assegura que o *pipeline* cumpre o rigor exigido por sistemas de natureza fiscal, garantindo zero perda informacional. Contudo, é válido salientar que essa ainda não é uma abordagem ideal, muito menos escalável. A natureza *ad-hoc* e manual da execução dos testes, além de demasiadamente custosa e propensa a erros, pode ser melhorada e explorada, buscando atender a métricas de cobertura de código, casos de teste de regressão que permitam testar melhorias de código e, principalmente, fornecer um modelo expansível e automatizado de execução dos mesmos.

Além disso, o desenvolvimento expôs desafios significativos inerentes à engenharia de dados em larga escala. A identificação dos gargalos de paginação do tipo JSONB e a degradação temporal causada pela necessidade do *PostgreSQL* de manter a consistência dos dados evidenciaram as limitações de ferramentas relacionais puras para armazenamento de formato bruto, bem como dos *trade-offs* de modelagem. Contudo, a análise detalhada desses comportamentos imprevisíveis constitui uma contribuição acadêmica e técnica relevante deste trabalho, balizando o entendimento sobre escolhas arquiteturais e fundamentando as diretrizes para a evolução contínua da ferramenta.

Em suma, o projeto atendeu de maneira satisfatória aos requisitos propostos, entregando um orquestrador resiliente, rastreável e flexível. A solução consolida uma base sólida que não apenas mitiga o esforço de manutenção de *software* perante a volatilidade da legislação tributária brasileira, mas também habilita a transformação de dados opacos em informações estruturadas prontas para consumo analítico e a aplicação direta de uma ideia que pode ser expandida para outros sistemas de mesmo gênero.

5.1 Trabalhos Futuros

Considerando as limitações mapeadas durante a avaliação do protótipo e o vasto potencial de expansão da arquitetura estabelecida, sugerem-se as seguintes frentes de pesquisa e desenvolvimento para o aprimoramento da solução:

- **Otimização do Motor de Persistência para Volumes Massivos:** Tendo em vista que a saturação de desempenho está atrelada ao custo de manutenção da consistência transacional do SGBD, sugere-se o desenvolvimento de um pipeline otimizado de carga. Este estudo futuro envolveria a adoção de *tabelas de estagiamento*⁷ sem índices para a recepção bruta dos registros EFD ICMS IPI via *bulk inserts*⁸, seguida pela consolidação assíncrona e particionamento dos dados nas tabelas finais do modelo relacional;
- **Testes de Estresse e Escalabilidade Horizontal:** Executar baterias de testes quantitativos com volumetria de produção em ambiente distribuído, com o objetivo de traçar curvas de escalabilidade linear do Orquestrador, avaliando o comportamento do gerenciamento concorrente de *workers* sob carga massiva e extrema;
- **Utilização de *parsing* e *loading* baseado em metadados em outros cenários:** Visando atestar a eficiência e manutenibilidade dessa abordagem, é sugerido que se faça o uso da mesma para outros formatos de documento em texto semiestruturado que requeiram a criação de um *parsing* dedicado e que tenham alta volatilidade. O objetivo aqui seria entender como essa abordagem pode ser aplicada em outros casos e quais os níveis de sucesso e mudanças necessárias a serem aplicadas;

⁷Estruturas temporárias de armazenamento usadas para receber dados brutos rapidamente, minimizando o custo de restrições de integridade e atualização de índices durante a carga inicial.

⁸Técnica de ingestão de dados que envia grandes volumes de registros em um único lote ou comando para o SGBD, reduzindo o custo computacional por linha inserida.

6 DECLARAÇÃO SOBRE USO DE INTELIGÊNCIA ARTIFICIAL

Em conformidade com as diretrizes de integridade acadêmica e transparência na pesquisa científica, o autor declara o uso de ferramentas de Inteligência Artificial Generativa — especificamente o modelo de linguagem Google Gemini — como tecnologia assistiva durante as etapas de ideação, redação, formatação e revisão deste Trabalho de Conclusão de Curso.

O emprego da referida ferramenta ocorreu sob supervisão e direcionamento estrito do autor, restringindo-se às seguintes frentes operacionais:

- **Geração Direta de Texto:** A ferramenta foi empregada para a consolidação textual do Resumo (e sua respectiva tradução para o *Abstract*) e para a redação desta própria seção de Declaração de Uso de Inteligência Artificial. Destaca-se que a geração ocorreu estritamente a partir das premissas, dados analíticos e direcionamentos estruturais fornecidos previamente pelo autor.
- **Consultoria Metodológica, Normativa e Ética:** Realização de consultas referentes à conformidade do documento com as regras da Associação Brasileira de Normas Técnicas (ABNT), abrangendo o refatoramento de citações e a obtenção de guias orientativos sobre o escopo de conteúdo exigido para determinadas seções do trabalho.
- **Revisão Gramatical e Refinamento Estilístico:** Lapidação do texto em língua portuguesa, auxiliando na adequação da sintaxe, eliminação de vícios de linguagem, garantia de coesão textual e elevação do vocabulário para manter o tom formal, objetivo e impessoal exigido pela escrita acadêmica.
- **Codificação LaTeX e Visualização de Dados:** A ferramenta atuou como suporte técnico na formatação estrutural do documento e na resolução de problemas de compilação, assegurando a conformidade com as normas da ABNT. O auxílio abrangeu a organização modular do código-fonte e a criação do *boxplot* utilizado na apresentação dos dados na seção 4.4.
- **Discussão Conceitual e Ideação:** Uso do modelo de linguagem como ferramenta de *brainstorming*, englobando a formulação de ideias de títulos para o trabalho e a organização em tópicos de determinadas seções — a exemplo da separação encontrada na seção de resultados.

Por fim, o autor reitera que a utilização destas ferramentas de Inteligência Artificial possuiu caráter consultivo e de apoio estrutural. Dessa forma, o autor atesta ter revisado integralmente o material gerado e assume a responsabilidade total, irrestrita e exclusiva sobre todo o conteúdo intelectual, analítico e técnico apresentado neste documento, bem como pela originalidade da pesquisa e pela garantia de ausência de plágio.

REFERÊNCIAS

- BRASIL. **Decreto nº 6.022, de 22 de janeiro de 2007**. Institui o Sistema Público de Escrituração Digital — SPED. Diário Oficial da União: seção 1, Brasília, DF, 22 jan. 2007. Disponível em: https://www.planalto.gov.br/ccivil_03/_ato2007-2010/2007/decreto/d6022.htm. Acesso em: 13 jul. 2026.
- BRASIL. **Modernização da relação Fisco-Contribuinte**. Sistema Público de Escrituração Digital (SPED), 2026. Disponível em: <https://www.gov.br/sped/pt-br/assuntos/modernizacao-da-relacao-fisco-contribuinte>. Acesso em: 13 jul. 2026.
- BRASIL. **Escrituração Fiscal Digital (EFD ICMS IPI)**. Sistema Público de Escrituração Digital (SPED), 2026. Disponível em: <https://www.gov.br/sped/pt-br/assuntos/escrituracoes-digitais/efd-icms-ipi>. Acesso em: 13 jul. 2026.
- BRASIL. **Guia Prático da Escrituração Fiscal Digital — EFD ICMS/IPÍ**: Versão 3.2.3. Sistema Público de Escrituração Digital (SPED), 2026. Disponível em: <https://www.gov.br/sped/pt-br/assuntos/escrituracoes-digitais/efd-icms-ipi/manuais-e-documentos-tecnicos/guia-pratico-efd-versao-3-2-3.pdf>. Acesso em: 13 jul. 2026.
- BRASIL. **Lei nº 5.152, de 25 de outubro DE 1966..** Dispõe sobre o Sistema Tributário Nacional e institui normas gerais de direito tributário aplicáveis à União, Estados e Municípios. Diário Oficial da União: seção 1, Brasília, DF, 27 out. 1966. Disponível em: https://www.planalto.gov.br/ccivil_03/leis/l5172compilado.htm. Acesso em: 13 jul. 2026.
- BRASIL. **Nota Técnica da Escrituração Fiscal Digital — EFD ICMS/IPÍ**: Versão 2026.001 v1.0. Sistema Público de Escrituração Digital (SPED), 2026. Disponível em: <https://www.gov.br/sped/pt-br/assuntos/escrituracoes-digitais/efd-icms-ipi/manuais-e-documentos-tecnicos/nt-2026-001-v1-0.pdf/>. Acesso em: 13 jul. 2026.
- CHIRUMAMILLA, Koteswara Rao. Metadata-driven ETL framework for accelerating cloud modernization in retail. **International Journal of Emerging Technologies and Engineering Research (IJEETR)**, v. 6, n. 3, p. 8106–8121, 2024. Disponível em: <https://ijeetr.com/index.php/ijeetr/article/view/184/170>. Acesso em: 14 jul. 2026.
- TAVARES, Saymon Souza; FERREIRA, Matheus Leandro. **NF-e Controller: sistema de auxílio contábil com centralização e gerenciamento de notas fiscais eletrônicas advindas da Secretaria da Fazenda**. 2022. Trabalho de Conclusão de Curso (Graduação) – Universidade do Extremo Sul Catarinense (UNESC), Criciúma, 2022. Disponível em: <http://repositorio.unesc.net/handle/1/10338>. Acesso em: 20 jul. 2026.
- BRASIL. Conselho Nacional de Política Fazendária. Comissão Técnica Permanente do ICMS. **Ato COTEPE/ICMS nº 44, de 7 de agosto de 2018**. Dispõe sobre as especificações técnicas da Escrituração Fiscal Digital — EFD. Diário Oficial da União: seção 1, Brasília, DF, 8 ago. 2018. Disponível em: <https://www.confaz.fazenda.gov.br/legislacao/atos/2018/ato-cotepe-icms-44-18>. Acesso em: 13 jul. 2026.

BRASIL. Ministério da Fazenda. Portal Nacional da Nota Fiscal Eletrônica. **Schemas XML NF-e - 010e_v.1.02**: NT 2025.002 v.1.40, NT 2026.002 v.1.0 e NT 2026.003 v.1.0. Brasília, DF: Ministério da Fazenda, 2026. Conjunto de arquivos XSD. Disponível em: <https://www.nfe.fazenda.gov.br/portal/exibirArquivo.aspx?conteudo=akib2DRpJN4=>. Acesso em: 19 jul. 2026.

BRASIL. **Perguntas Frequentes da Escrituração Fiscal Digital — EFD ICMS/IPI**: Versão 7.6. Receita Federal do Brasil, Sistema Público de Escrituração Digital (SPED). Disponível em: <https://www.gov.br/receitafederal/pt-br/assuntos/empresas-e-negocios/sped/efd-icms-ipi/perguntas-frequentes/perguntas-frequentes-7-6.pdf>. Acesso em: 10 ago. 2026.